NSDE 框架技术说明文档：基于神经随机微分方程的期权定价

这份文档旨在从计算机科学与金融工程交叉的角度，详尽阐述 **NSDE (Neural Stochastic Differential Equations)** 期权定价框架的技术实现、模型设计、数学原理及实验细节。

1. 框架概述 (Framework Overview)

本框架提出了一种**混合神经随机微分方程 (Hybrid NSDE)** 模型，用于捕捉金融市场中复杂的动力学特征（如波动率聚集、杠杆效应及情绪溢出）。该框架的核心思想是将经典的 **Heston 模型** 作为物理先验（Base Model），并利用神经网络（Neural Networks）学习未被经典模型捕捉的残差动力学（Residual Dynamics）。

1.1 核心设计逻辑

- **动力学层面的融合**：不同于传统的“神经网络直接预测价格”，本框架在微分方程层面进行参数化，保留了定价过程的连续性与物理可解释性。
- **情绪因子整合**：通过将隐空间的情绪特征 z 注入扩散（Diffusion）项，模型能够动态调整资产价格的演化路径。

2. 数学背景与模型动力学 (Mathematical Framework)

2.1 连续时间动力学描述

在风险中性测度 $\mathbb{Q}$ 下，标的资产价格 S_t 与随机方差 v_t 的演化遵循以下耦合的随机微分方程：

$$\begin{cases} dS_t = rS_t dt + \sigma_S(S_t, v_t, r, t, z) S_t dW_t^S \\ dv_t = \mu_v(S_t, v_t, r, t, z) dt + \sigma_v(S_t, v_t, r, t, z) \sqrt{v_t} dW_t^v \end{cases}$$

关于漂移项 rS_t 的说明：
根据无套利定价原理，资产在风险中性下的期望收益率必须等于无风险利率。因此，本框架显式硬编码 S_t 的漂移项，不使用神经网络扰动，以确保物理定律的一致性。

2.2 混合残差建模与残差缩放因子 (λ)

为保证模型在训练初期的数值稳定性并约束神经网络的搜索空间，我们定义了**残差缩放因子** λ。在每一时间步 t，SDE 的系数通过以下算子合成：

- **资产扩散项修正：**

$$\sigma_S(\mathbf{x}_t) = \text{clamp}\left(\sqrt{v_t} + \lambda \cdot \text{NN}_{\sigma_S}(\mathbf{x}_t;\theta), \epsilon, \sigma_{max}\right)$$

- **波动率漂移项修正：**

$$\mu_v(\mathbf{x}_t) = \kappa(\theta - v_t) + \lambda \cdot \text{NN}_{\mu_v}(\mathbf{x}_t;\theta)$$

- **波动率扩散项修正：**

$$\sigma_v(\mathbf{x}_t) = \text{clamp}\left(\sigma_{Heston} + \lambda \cdot \text{NN}_{\sigma_v}(\mathbf{x}_t;\theta), \epsilon, \sigma_{max}\right)$$

其中 $\mathbf{x}_t = [S_t/S_0, v_t, r, t, z]$，$\theta$ 为神经网络参数。
λ 的数学意义： 它充当了基准物理模型与神经修正项之间的**混合系数（Mixing Coefficient）**。通过限制 λ（如 `0.3`），我们实际上是在以 Heston 模型为中心的邻域内寻找最优动力学解，防止神经网络在训练初期由于随机梯度导致的路径崩塌。

2.3 数值离散化：Euler-Maruyama 方案

为了在计算机中实现上述连续 SDE，我们采用时间步长 $\Delta t = T/N$ 进行离散化。每一时刻 t 到 $t+1$ 的状态转移方程如下：

- **资产价格演化步：**

$$S_{t+1} = S_t + rS_t\Delta t + \sigma_S(\mathbf{x}_t)S_t\sqrt{\Delta t}Z_t^S$$

- **波动率方差演化步：**

$$v_{t+1} = \text{Softplus}\left(v_t + \mu_v(\mathbf{x}_t)\Delta t + \sigma_v(\mathbf{x}_t)\sqrt{v_t}\sqrt{\Delta t}Z_t^v\right)$$

其中 Z_t^S 与 Z_t^v 为遵循 $\mathbb{Q}$ 测度下相关系数为 ρ 的标准正态分布随机数。 `Softplus` 算子的引入确保了 v_t 在数值模拟过程中的非负性，并为反向传播提供了连续的梯度流。

3. 训练机制拆解：模型到底在学什么?

理解神经网络训练的关键在于看清**参数（Parameters）**、**前向模拟（Forward）** 与 **反向传播（Backward）** 的闭环。

3.1 什么是“可学习”的?

本框架并不直接学习期权价格，而是学习 **4 个 MLP 网络的权重 W 和偏置 b**（在公式中统一记为参数向量 θ）。这些参数构成了动力学系统的“控制器”：

- 如果 Heston 模型在低波动率区域系统性地低估了价格，反向传播会调整网络权重 θ，使得在相应状态下输出正的残差，从而通过增大模拟路径的扩散范围来提升最终定价。

3.2 训练闭环：反向传播通过 SDE (BPTT) 的详尽计算过程

本模型的核心挑战在于梯度必须穿越蒙特卡洛（MC）路径。对于 MSE 损失函数 $\mathcal{L}$，梯度的计算涉及以下四个严密的求导环节：

Step 1: 标量损失层级的微分 (The Loss Scalar Gradient)

首先计算目标函数 $\mathcal{L}$ 对预测价格 C_{pred} 的偏导数。
标量损失定义：在一次批处理（Batch）中，损失函数 $\mathcal{L}(\theta) = \frac{1}{B}\sum_{i=1}^{B}(C_{pred,i} - C_{market,i})^2$ 将高维的预测误差压缩为一个单一的**实数标量**。

$$\frac{\partial \mathcal{L}}{\partial \theta} = \underbrace{\frac{2}{B}\sum_{i=1}^{B}(C_{pred,i} - C_{market,i})}_{\text{标量误差方向}} \cdot \frac{\partial C_{pred}}{\partial \theta}$$

- $\mathcal{L}$：反映了全量权重的总代价（Cost）。
- **标量性质**：只有当损失是一个标量时，梯度 $\nabla_\theta \mathcal{L}$ 才代表了 θ 参数空间中最陡峭的下降方向。这里确立了全局误差的符号：若 $C_{pred} > C_{market}$，导数为正，引导 θ 向减少输出的方向调整。

Step 2: 定价对终端价格状态的求导 (Payoff Gradient)

由于 $C_{pred} = e^{-rT}\frac{1}{M}\sum \Phi(S_T^{(m)})$，模型价对网络参数的偏导数转化为期望的偏导数：

$$\frac{\partial C_{pred}}{\partial \theta} = e^{-rT}\frac{1}{M}\sum_{m=1}^{M}\left(\frac{\partial \Phi(S_T)}{\partial S_T}\cdot\frac{\partial S_T}{\partial \theta}\right)^{(m)}$$

- M：代表蒙特卡洛模拟生成的独立路径总数（如训练时取 1000）。

- $\frac{\partial \Phi(S_T)}{\partial S_T}$：这是**行权收益函数对终端价格的求导**。对于 Call Option，当 $S_T > K$ 时导数为 1，否则为 0。它告诉模型：只有“落入价内”的路径梯度才能回传。

Step 3: 终端状态对神经网络输出的求导 (Temporal Chain)

这是 BPTT 的核心。通过链式法则，终端价格 S_T 对参数 θ 的依赖被分解为每一时间步的累积：

$$\frac{\partial S_T}{\partial \theta} = \sum_{t=0}^{T-1} \underbrace{\left(\prod_{k=t+1}^{T-1} \frac{\partial S_{k+1}}{\partial S_k} \right)}_{\text{路径传递矩阵}} \cdot \frac{\partial S_{t+1}}{\partial \mathrm{NN}_t} \cdot \frac{\partial \mathrm{NN}_t}{\partial \theta}$$

- $\frac{\partial S_{t+1}}{\partial \mathrm{NN}_t}$：这是**状态对神经修正项的求导**。它描述了神经网络输出的微调如何转化为价格的瞬时偏移：

$$\frac{\partial S_{t+1}}{\partial \mathrm{NN}_t} = \lambda \cdot S_t \sqrt{\Delta t} Z_t^S$$

  - **物理直观**：该导数揭示了神经网络如何利用当前的随机冲击 Z_t^S 作为“杠杆抓手”。其方向由随机噪声引导，决定了模型如何利用随机性来修正定价偏差。
- $\frac{\partial S_{k+1}}{\partial S_k}$：这是**状态转移梯度**。它描述了 t 时刻产生的路径偏移如何传递到下一时刻：

$$\frac{\partial S_{k+1}}{\partial S_k} = \underbrace{\left(1 + r\Delta t + \sigma_{total}\sqrt{\Delta t} Z_k^S\right)}_{\text{被动继承}} + \underbrace{S_k \sqrt{\Delta t} Z_k^S \cdot \left(\frac{\partial \sigma_{base}}{\partial S_k} + \lambda \frac{\partial \mathrm{NN}_k}{\partial S_k} \right)}_{\text{反馈效应}}$$

  - **数学本质**：由两部分组成。前者源于资产价格自身的基数效应（Direct Scaling），后者源于价格变动通过输入特征（Moneyness S_k/S_0）回传给神经网络后产生的反馈环。

Step 4: 神经网络输出对内部权重的求导 (Network Gradient)

最后回到神经网络本身：

$$\frac{\partial \mathrm{NN}_t}{\partial \theta}$$

这是标准的 MLP 求导。它说明了如何微调权重 θ 来精确控制 t 时刻网络产生的残差输出。

4. 模型架构设计 (Model Architecture)

4.1 神经网络组件 (FlexibleMLP)

- **LayerNorm**：加速收敛，稳定每一时刻网络输出的量级。
- **Softplus 约束**：对于 v_t 状态量的更新逻辑：

$$v_{t+1} = \frac{1}{\beta} \log(1 + \exp(\beta \cdot v_{t+1}^{raw}))$$

  该算子确保波动率项在 $v_t \to 0$ 边界处平滑可导，解决“零波动率死区”问题。

4.2 特征工程

- **Moneyness** (S_t/S_0)：消除价格绝对尺度的影响。
- **情感注入**：由 VAE 提取的情绪因子 z 参与动力学修正。

5. 模型配置与超参数 (Hyperparameters)

5.1 神经网络与模拟参数

- **隐藏层维度 (`hidden_dims`)**: `[32, 32]` 。
- **残差缩放因子 (`residual_scale`)**: `0.3` 。
- **离散步数 (`n_steps`)**: `50` 步。
- **路径数量 (`n_paths`)**: 训练阶段使用 `1000` 条路径配合对偶变量法。

5.2 优化参数

- **学习率 (`lr`)**: `1e-3` 。
- **批大小 (`batch_size`)**: `64` 。
- **损失函数 (`loss_type`)**: 核心优化目标为 **MSE (Mean Squared Error)**。

6. 数值模拟与评估指标

6.1 方差减少：对偶变量法 (Antithetic Variates)

生成对称路径 $\mathbf{Z}_{paths} = [\mathbf{Z}, -\mathbf{Z}]$。这在数学上通过构造负相关变量，消除了蒙特卡洛积分中的一阶偏差，稳定了 $\frac{\partial C_{pred}}{\partial \theta}$ 的估值。

6.2 评估核心指标

采用 **MAE, RMSE, MAPE**。

> **结论**：本 NSDE 框架将神经网络作为嵌入式模块注入到随机动力学系统中。通过在蒙特卡洛演化路径上进行端到端的梯度优化，模型实现了对金融衍生品复杂定价特征的高维度、动态化捕捉。